

WolfSSL Cheat Sheet

~By former BWSI (2023) students~

Wolf SSL is a helpful library when encrypting from the bootloader or hashing your packets for authenticity in C. Depending on how you design your program, the codes below might help you with your Design Challenge. Check your library implementation and ensure that what you intend to include is defined as included.

Prerequisite:

Make sure to include WolfSSL in your program.

```
#include "wolfssl/wolfcrypt/settings.h"
#include "wolfssl/wolfcrypt/sha.h"
#include "wolfssl/wolfcrypt/aes.h"
#include "wolfssl/wolfcrypt/rsa.h"
```

SHA:

Below is a default way to use hash. It's the simplest, best way to use them.

Visit https://www.wolfssl.com/documentation/manuals/wolfssl/sha_8h.html for more.

```
wc\_Sha256Hash(const byte* data, word32 len, byte * hash)
```

Example: Generating myHash and sending it to the board. <TESTED & WORKS>

```
// Don't forget your header files
initialize_uart(UART0);

const char* str = "This is a secret"; // Make a string
uint8_t size = strlen(str); // Get the size

byte data[size]; // Put the string into byte
memcpy(data, str, size); // Copy it

byte myHash[32];
wc_Sha256Hash(data, size, myHash); // Hash it
for(int i = 0; i < 32; i++){uart_write_hex(UART0, myHash[i]);}
// Write it
```

AES:

Visit https://www.wolfssl.com/documentation/manuals/wolfssl/group_AES.html for more.

```
wc\_AesInit(Aes * aes, void * heap, int devId);
```

```
wc\_AesSetIV(Aes * aes, const byte * iv);
```

```
wc\_AesSetKey(Aes * aes, const byte * key, word32 len, const
byte * iv, int dir);
```

AES CBC:

```
wc\_AesCbcDecrypt(Aes * aes, byte * out, const byte * in, word32 sz);
```

```
wc\_AesCbcEncrypt(Aes * aes, byte * out, const byte * in, word32 sz);
```

AES GCM:

```
wc\_AesGcmSetKey(Aes *aes, const byte *key, word32 len);
```

```
wc\_AesGcmEncrypt(Aes *aes, byte *out, const byte *in, word32 sz, const byte *iv, word32 ivSz, byte *authTag, word32 authTagSz, const byte *authIn, word32 authInSz);
```

```
wc\_AesGcmDecrypt(Aes *aes, byte *out, const byte *in, word32 sz, const byte *iv, word32 ivSz, const byte *authTag, word32 authTagSz, const byte *authIn, word32 authInSz);
```

Example: AES CBC decryption, where I read and write on the board. <TESTED & WORKS>

```
#define KEY "arthurrobinsoniv"
// Don't forget your header files
initialize_uarts(UART0);
Aes dec; // Decrypt of Aes *aes
Aes enc; // Encrypt of Aes *aes

uint8_t iv[16] = "arthurrobinsoniv";

char ciphertext[] =
{0xe6,0x4d,0x52,0x79,0x38,0xe4,0x34,0xba,0x6b,0x27,0xd3,0xdf,0xdd,0x3a,0xa9,0x8d,0x6e,0xfd,0xd4,0x17,0x16,0xb0,0xc4,0x89,0x34,0xe7,0x5a,0xee,0x25,0xb6,0xea,0xdb,0x1a,0x83,0xdd,0xcf,0x19,0xc5,0xd2,0xf9,0xf3,0xc7,0x71,0xa2,0x5b,0x25,0xd4,0x11};

uint8_t size = sizeof(ciphertext);

wc_AesSetIV(&dec, iv); // Set IV

wc_AesSetKey(&dec, KEY, 16, iv, AES_DECRYPTION); //Set KEY

char plaintext[size]; // Define plaintext
wc_AesCbcDecrypt(&dec, plaintext, ciphertext, size); // Decrypt

for (int i = 0; i < size; i++){uart_write_hex(UART0,
plaintext[i]);} // Print plaintext
```

RSA:

[`wc\_InitRsaKey`](#) (RsaKey *key, void *heap)

[`wc\_RsaPublicEncrypt`](#) (const byte *in, word32 inLen, byte *out, word32 outLen, RsaKey *key, WC_RNG *rng)

[`wc\_RsaPrivateDecrypt`](#) (const byte *in, word32 inLen, byte *out, word32 outLen, RsaKey *key)

[`wc\_RsaSSL\_Sign`](#) (const byte *in, word32 inLen, byte *out, word32 outLen, RsaKey *key, WC_RNG *rng)

[`wc\_RsaSSL\_Verify`](#) (const byte *in, word32 inLen, byte *out, word32 outLen, RsaKey *key)

Example: Verifying Signature via RSA. <TESTED & WORKS>

```
initialize_uarts(UART0);
// Imported variables
RsaKey pub;
// Note, the wolfssl original RsaKey rng is not recommended
#include "key.h" //Includes KEY

// Definitions
int ret;
byte output[256];
byte input[] = {0xb6, 0x9a, 0xf8, 0xe2, 0x51, 0xed, 0xf4, 0x1c,
0x6f, 0xad, 0xc4, 0x3c, 0x11, 0xbc, 0x12, 0x26, 0xdb, 0xd1,
0x97, 0xa7, 0xc3, 0xb8, 0x99, 0x2c, 0xc5, 0x3b, 0x88, 0x98,
0x26, 0xd3, 0xfd, 0x10, 0x3d, 0x71, 0x23, 0xa8, 0xd6, 0x1f,
0x5f, 0xb2, 0x38, 0x77, 0x7a, 0x45, 0xc1, 0xde, 0x41, 0xd,
0xe2, 0xa, 0x8, 0x72, 0x75, 0xe5, 0x6b, 0x7b, 0xa2, 0x4, 0x7b,
0x94, 0xc9, 0x86, 0x93, 0x46, 0xff, 0xa2, 0xd9, 0xcd, 0x0,
0x11, 0x49, 0x80, 0x21, 0xb7, 0xf2, 0xe9, 0x83, 0xd4, 0xb1,
0xc3, 0x8c, 0x96, 0x58, 0x46, 0x35, 0xb0, 0xd2, 0xfb, 0xe,
0xab, 0x6c, 0xaf, 0x11, 0xa8, 0x29, 0x1f, 0x9d, 0xc3, 0x1a,
0xb, 0x24, 0xb3, 0x6c, 0xb7, 0xa2, 0x7, 0x33, 0xa8, 0xc9, 0x95,
0x5e, 0xfa, 0x54, 0xea, 0x92, 0x72, 0x3d, 0xe7, 0xff, 0x18,
0x11, 0xe2, 0xc6, 0x79, 0xd9, 0x31, 0xf8, 0x63, 0x3d, 0xc,
0xeb, 0x62, 0x6d, 0x96, 0x3a, 0x68, 0xbc, 0x4f, 0x11, 0x3a,
0xd2, 0x13, 0xc3, 0x8f, 0xac, 0xdb, 0xbd, 0xc5, 0xb0, 0x35,
0x39, 0xed, 0xb8, 0xba, 0x4b, 0xe8, 0x47, 0x7e, 0x55, 0xc4,
0xeb, 0x57, 0xec, 0x8b, 0xbe, 0x7d, 0xe3, 0x5b, 0xe2, 0xb2,
0x30, 0x51, 0xd8, 0x3c, 0x20, 0x65, 0xd5, 0x96, 0xf4, 0x6b,
0xf, 0xcf, 0xe2, 0xf0, 0xa3, 0x69, 0xdf, 0xb4, 0x5c, 0x86,
0xf2, 0xed, 0xb0, 0xe4, 0xe7, 0xf2, 0xc9, 0x60, 0xd4, 0x94,
0x6b, 0x35, 0xd3, 0x4a, 0x67, 0xb0, 0xbd, 0x89, 0x5d, 0x59,
```

```
0x1c, 0x18, 0x7b, 0x1e, 0x22, 0xaa, 0x72, 0x72, 0xcd, 0x63,
0x93, 0x15, 0x71, 0xf1, 0x30, 0x73, 0xfc, 0xc4, 0xc, 0x92,
0x55, 0xae, 0x1d, 0x50, 0x73, 0x6e, 0x8d, 0xe9, 0x62, 0x14,
0xf2, 0x89, 0x7b, 0xb2, 0xef, 0xbd, 0xe, 0xe0, 0xd, 0x60, 0x6a,
0x78, 0x63, 0x7, 0x66, 0xe4};

wc_InitRsaKey(&pub, NULL);

int i = 0;
wc_RsaPublicKeyDecode(KEY, &i, &pub, sizeof(KEY));

ret = wc_RsaPSS_Verify(input, sizeof(input), output,
sizeof(output), WC_HASH_TYPE_SHA256, WC_MGF1SHA256, &pub);

if(ret <= 0){// Verifying
    uart_write_str(UART0, "Successfully Verified
Signature!\n");
}
else{
    uart_write_str(UART0, "ERROR");
}
```



Thank you for using this cheat sheet!!!

WolfSSL is so cool 🐺🐺🐺